



Universidade Estadual de Mato Grosso do Sul

Ciência da Computação

Banco de Dados

SQL - parte 1

Prof. Nilton
nilton@comp.uems.br

S Q L

a linguagem de banco de dados

- Muitos SGBDs relacionais comerciais vêm com uma linguagem de alto nível declarativa
 - Especifica-se o resultado desejado e o SGBD resolve a otimização e a execução
- Início dos anos de 1970 a IBM desenvolveu a linguagem **SEQUEL** (*Structured English QUery Language*) para o SYSTEM R (SGBD experimental)
 - Demonstrar a viabilidade do modelo relacional de Edgar Frank Codd
- Em 1977 passou a ser chamada de **SQL** (*Structured Query Language*)

O padrão SQL

- Existe um esforço contínuo por estabelecimento de um padrão SQL (ANSI)
 - O primeiro padrão (1986) - **SQL1 ou SQL-86**
 - DDL e DML
 - 1989 extensão para integridade referencial e restrições de integridade
 - O segundo padrão (1992) - **SQL2 ou SQL-92**
 - Facilidade de esquema e administração de dados
 - Melhoramento no DDL e DML
 - Triggers, asserções (debug), ferramentas descoberta conhecimento
 - Em 1999 definiu-se o padrão **SQL3 ou SQL-1999**
 - Gerenciamento de objetos (encapsulamento, subtipos, herança, polimorfismo)
 - Em 2003 (**SQL-2003**) e 2006 (**SQL-2006**)
 - Introdução a características relacionadas a XML
 - Em 2008 (**SQL-2008**)
 - Mais recursos de banco de dados de objetos

S Q L

características e instruções

- Instruções padronizadas na mesma nomenclatura e formato para diferentes SGBD
- Migração de um SGBD para outro não requer grandes mudanças
- Portabilidade entre plataformas
- As categorias de instruções de SQL
 - **DDL**: *Data Definition Language*
 - **DML**: *Data Manipulation Language*
 - **DCL**: *Data Control Language*
 - **TCL**: *Transact Control Language*

S Q L

instruções

- **DDL:** definição e manutenção das estruturas do banco de dados, tais como a criação do próprio banco de dados e das tabelas que compõe, além das relações entre as tabelas e os objetos do banco de dados.
- **DML:** consultas, inserções, exclusões e alterações em um ou mais registros, de uma ou mais tabelas.
- **DCL:** controle dos privilégios de usuários, de forma que o administrador do banco de dados possa determinar o nível de acesso de um usuário aos objetos do banco de dados, concedendo privilégios ou retirando esse acesso e revogando os privilégios.
- **TCL:** controle de transações, consideradas o conjunto de uma ou mais operações DML realizadas no banco de dados.

S Q L

instruções

- **DDL**

- Create: criação de estruturas de objetos do BD
- Alter: alteração da estrutura de objetos do BD
- Drop: eliminação da estrutura de objetos do BD

- **DML**

- Insert: inserção de dados
- Update: alteração de dados
- Delete: exclusão de dados
- Select: consulta de dados

S Q L

instruções

- **DCL**

- Grant: atribuição de privilégios aos usuários do BD
- Revoke: revogação de privilégios dos usuários do BD
- Deny: impedir um privilégio aos usuários do BD

- **TCL**

- Commit: confirmação das manipulações
- Rollback: desistência das manipulações

DDL – SQL2

- Um **esquema** é uma área que agrupa os objetos de um BD, tais como tabelas, restrições, visões, domínios, autorizações, índices, etc, pertencentes a um determinado usuário/aplicação.
- Caso um esquema não seja criado os objetos são colocados numa área comum para todos usuários/aplicações.
- **CREATE SCHEMA** Empresa **AUTHORIZATION** Jsilva;
- **CREATE DATABASE** Empresa;

DDL – SQL2

- **CREATE TABLE**
- **ALTER TABLE**
- **DROP TABLE**

O Banco de Dados Empresa

- Empregado(nominic, nomint, nomefim, numat, dtnasc, ender, sexo, salario, nsuper, ndepto)
- Departamento (nomedep, numdep, nchefe, dtinchef)
- Projeto (nomeproj, numproj, locproj, ndep)
- Dependente (numemp, depend, sexo, dtnasc, tipo)
- LocalDepto (numdep, locdep)
- TrabalhaEm (numemp, nproj, horas)

CREATE TABLE

- Para especificar uma nova tabela e atributos
- Para cada atributo: nome, tipo de dados + domínio, restrições (atributo, chave, integridade da entidade, integridade referencial)
- NULL permitido, quando não constar nas restrições
- O usuário deve ter privilégios, além de uma área de armazenamento dos dados

CREATE TABLE

```
CREATE TABLE [esquema.]tabela  
(nome do atributo tipo do dado [DEFAULT expr]  
[restrição do atributo],  
...,  
[restrição da tabela]);
```

- **esquema**: se omitido a tabela será criada no esquema corrente
- **tabela**: nome da tabela
- **nome do atributo**: onde o dado será armazenado
- **tipo do dado**: tipo do dado e tamanho do atributo
- **DEFAULT expr**: valor default quando dado inserido for omitido
- **restrição do atributo**: NOT NULL ou UNIQUE
- **restrição da tabela**: PRIMARY KEY, FOREIGN KEY

CREATE TABLE

tipos de dados

- Numéricos:
 - `INT, INTEGER, SMALLINT, FLOAT, REAL, DOUBLE PRECISION, DECIMAL(i, d), NUMERIC(i, d)`
- Cadeias de caracteres:
 - `CHAR[ACTER] (n), VARCHAR(n), CHAR[ACTER] VARYING(n)`
- Cadeias de bits:
 - `BIT [VARYING] (n)`
- Datas:
 - `DATE /*YYYY-MM-DD*/, YEAR, MONTH, DAY,`
 - `TIME /*HH:MM:SS*/, HOUR, MINUTE, SECOND`
 - `TIMESTAMP /* data, hora, fração de segundo */`

CREATE TABLE

Empregado

```
CREATE TABLE Empregado (  
    nominic    VARCHAR(15)    NOT NULL,  
    nomint     CHAR(1),  
    nomefim    VARCHAR(15)    NOT NULL,  
    numat      CHAR(4)        NOT NULL,  
    dtnasc     DATE,  
    ender      VARCHAR(30),  
    sexo       CHAR(1),  
    salario    DECIMAL(10,2),  
    nsuper     CHAR(4),  
    ndepto     INT            NOT NULL,  
    PRIMARY KEY (numat),  
    FOREIGN KEY (nsuper) REFERENCES Empregado(numat),  
    FOREIGN KEY (ndepto) REFERENCES Departamento(numdep)  
);
```

CREATE TABLE

Departamento

```
CREATE TABLE Departamento (  
    nomedep          VARCHAR(15)    NOT NULL,  
    numdep           INT             NOT NULL,  
    nchefe           CHAR(4) ,  
    dtinchef         DATE ,  
    PRIMARY KEY      (numdep) ,  
    UNIQUE           (nomedep) ,  
    FOREIGN KEY      (nchefe) REFERENCES Empregado (numat)  
);
```

- **UNIQUE** como restrição de tabela, podendo permitir a combinação de atributos com valores diferentes. Lembre-se que **UNIQUE** diferencia de **PRIMARY KEY** por permitir valores nulos.

CREATE TABLE

LocalDeppto

```
CREATE TABLE LocalDeppto (  
    numdep          INT          NOT NULL,  
    locdep          VARCHAR(15)  NOT NULL,  
    PRIMARY KEY (numdep, locdep),  
    FOREIGN KEY (numdep) REFERENCES Departamento (numdep)  
);
```


CREATE TABLE Projeto

```
CREATE TABLE Projeto (  
    nomeproj      VARCHAR(20)    NOT NULL,  
    numproj       INT            NOT NULL,  
    locproj       VARCHAR(15),  
    ndep          INT            NOT NULL,  
    PRIMARY KEY   (numproj),  
    UNIQUE        (nomeproj),  
    FOREIGN KEY   (ndep) REFERENCES Departamento (numdep)  
);
```

CREATE TABLE

TrabalhaEm

```
CREATE TABLE TrabalhaEm (  
    numemp          CHAR(4)          NOT NULL,  
    nproj           INT              NOT NULL,  
    horas           DECIMAL(3,1)     NOT NULL,  
    PRIMARY KEY (numemp, nproj),  
    FOREIGN KEY (numemp) REFERENCES Empregado(numat),  
    FOREIGN KEY (nproj) REFERENCES Projeto(numproj)  
);
```

CREATE TABLE

Dependente

```
CREATE TABLE Dependente (  
    numemp          CHAR(4)          NOT NULL,  
    depend          VARCHAR(15)      NOT NULL,  
    sexo            CHAR(1),  
    dtnasc          DATE,  
    tipo            VARCHAR(8)  
    PRIMARY KEY (numemp, depend),  
    FOREIGN KEY (numemp) REFERENCES Empregado(numat)  
);
```

CREATE TABLE

- Pode ser também
 - **CREATE TABLE Empresa.Empregado ...**
- Uma outra forma é apresentada dando nomes às restrições de integridade, com o estabelecimento de restrições adicionais: as qualificações

CREATE TABLE

Empregado – com qualificação

```
CREATE TABLE Empregado (  
    ...  
    ndepto    INT          NOT NULL          DEFAULT 1,  
    CONSTRAINT empchave  
        PRIMARY KEY (numat),  
    CONSTRAINT empsuper  
        FOREIGN KEY (nsuper) REFERENCES Empregado (numat)  
            ON DELETE SET NULL  
            ON UPDATE CASCADE,  
    CONSTRAINT empdepto  
        FOREIGN KEY (ndepto) REFERENCES Departamento (numdep)  
            ON DELETE SET DEFAULT  
            ON UPDATE CASCADE  
);
```

- **CONSTRAINT empchave:** nome da restrição para chave primária dentro do esquema. Se não for informado será usado o nome do atributo numat.

Ações de gatilho referencial ON DELETE

ON DELETE { NO ACTION, RESTRICT, CASCADE, SET NULL, SET DEFAULT }

- **NO ACTION:** é o padrão quando ON DELETE não está especificado. Ao excluir um registro pai e registros filhos ficarão órfãos, o registro deletado será revertido.
- **RESTRICT:** é o mesmo que NO ACTION, porém a exclusão será impedida caso resulte em filhos órfãos.
- **CASCADE:** ao excluir um registro pai todos os registros filhos órfãos serão também excluídos.
- **SET NULL:** ao excluir um registro pai as chaves estrangeiras dos registros filhos órfãos receberão o valor NULL quando possível
- **SET DEFAULT:** ao excluir um registro pai as chaves estrangeiras dos registros filhos órfãos receberão o valor DEFAULT quando possível

Ações de gatilho referencial ON UPDATE

ON UPDATE { NO ACTION, RESTRICT, CASCADE, SET NULL, SET DEFAULT }

- **NO ACTION:** é o padrão quando ON UPDATE não está especificado. Ao alterar a chave primária do registro pai com registros filhos, a atualização falhará.
- **RESTRICT:** nenhuma mudança haverá na tabela pai.
- **CASCADE:** ao alterar a chave primária do registro pai todos os registros filhos nas chaves estrangeiras serão alterados.
- **SET NULL:** ao alterar a chave primária do registro pai as chaves estrangeiras dos registros filhos receberão o valor NULL caso aceitável
- **SET DEFAULT:** ao alterar a chave primária do registro pai as chaves estrangeiras dos registros filhos receberão o valor DEFAULT caso aceitável

CREATE TABLE

Departamento

```
CREATE TABLE Departamento (  
    ...  
    nchefe      CHAR(4) NOT NULL DEFAULT '8886',  
    ...  
    CONSTRAINT depchave  
        PRIMARY KEY (numdep),  
    CONSTRAINT depsecchave  
        UNIQUE (nomedep),  
    CONSTRAINT depchefe  
        FOREIGN KEY (nchefe) REFERENCES Empregado (numat)  
        ON DELETE SET DEFAULT  
        ON UPDATE CASCADE  
);
```


CREATE TABLE

LocalDeppto

```
CREATE TABLE LocalDeppto (  
    ...  
    PRIMARY KEY (numdep, locdep),  
    FOREIGN KEY (numdep) REFERENCES Departamento (numdep)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

ALTER TABLE

- Para alterar a estrutura de uma tabela, incluindo ou redefinindo um atributo ou restrição de uma tabela
- Havendo registros quando um atributo for acrescentado à tabela, o novo atributo será inicialmente nulo para todos os registros existentes
- A alteração do tipo ou tamanho do atributo terá efeito somente para os novos registros

ALTER TABLE ADD

```
ALTER TABLE [esquema.]tabela  
ADD(atributo tipo de dado [restrições], ...);
```

- **esquema:** se omitido a tabela será criada no esquema corrente
- **tabela:** nome da tabela
- **ADD:** adicionar atributos ou restrições à tabela
- **atributo:** nome do atributo que será acrescentado
- **tipo de dado:** tipo e tamanho do atributo que será acrescentado
- **restrições:** especificação das restrições para o atributo acrescentado

ALTER TABLE

ALTER/DROP

```
ALTER TABLE [esquema.]tabela  
ALTER/DROP (atributo especificações);
```

- **esquema:** se omitido a tabela será criada no esquema corrente
- **tabela:** nome da tabela
- **ALTER:** modificar atributos ou restrições à tabela
- **DROP:** excluir atributos ou restrições da tabela
- **atributo:** nome do atributo que será alterado
- **especificações:** tipo de dado, tamanho do atributo, restrições, etc

ALTER TABLE

- Para adicionar atributo:

- `ALTER TABLE Empregado ADD cargo VARCHAR(12);`

- Para fazer alteração:

- `ALTER TABLE Empregado ALTER COLUMN cargo TO nome_cargo;`
 - Altera o nome de um atributo
- `ALTER TABLE Empregado ALTER nchefe DROP DEFAULT;`
 - Remove uma cláusula default existente
- `ALTER TABLE Empregado ALTER nchefe SET DEFAULT '3334';`
 - Define uma nova cláusula default
- `ALTER TABLE Empregado MODIFY nome_cargo VARCHAR(30);`
 - Altera o tamanho de um atributo
- `ALTER TABLE Empregado RENAME TO Empregados;`
 - Altera o nome de uma tabela

ALTER TABLE

- Para eliminar atributo:

- `ALTER TABLE Empregado DROP endereco CASCADE;`
 - Exclui também os atributos filhos
- `ALTER TABLE Empregado DROP endereco RESTRICT;`
 - Não permite a exclusão se houver atributo filho

- Para alterar restrições:

- `ALTER TABLE Empregado DROP CONSTRAINT empsuper CASCADE;`
 - A exclusão da restrição somente é permitida se houver um nome dado anteriormente a ela
- `ALTER TABLE Empregado ADD CONSTRAINT ... ;`

DROP SCHEMA e DROP TABLE

- Para remover um esquema:
 - `DROP SCHEMA Empresa CASCADE;`
- Para remover a tabela e sua definição
 - `DROP TABLE Dependente CASCADE;`
- Em vez de CASCADE pode ser RESTRICT
 - CASCADE: remove tudo relativo ao esquema/tabela;
 - RESTRICT: remove se não houver nenhum elemento/restrição

DML – SQL2

- **INSERT**
- **DELETE**
- **UPDATE**
- **SELECT**

Comando **INSERT**

- Acrescenta um registro na tabela.
- Atributos na mesma ordem que em CREATE TABLE.

```
INSERT INTO Empregado VALUES (  
    'Ricardo', 'K', 'Marini', 6532,  
    30.12.72, 'Carmo 412', 370, '9876', 4);
```

Comando INSERT

- Segunda forma: especificar nomes de atributos explícitos que correspondam aos valores no comando
- Os outros são preenchidos com NULL

```
INSERT INTO Empregado(  
    nominic, nomefim, numat, ndepto)  
VALUES ('Roberto', 'Gomes', 9807, 4);
```

Comando INSERT

```
INSERT INTO Empregado (  
    nominic, nomefim, ndepto)  
VALUES ('Mário', 'Gomes', 2);
```

- Rejeita porque faltou especificar o campo numat que é um campo NOT NULL
- Adicionalmente, o departamento 2 não existe

Tabelas temporárias

```
CREATE TABLE Info_Depto
  (nomedepto  VARCHAR(15),
   numemps    INTEGER,
   saltot     INTEGER);
...
INSERT INTO Info_Depto(nomedepto, numemps, saltot)
  SELECT      nomedep, COUNT(*), SUM (salario)
  FROM        Departamento, Empregado
  WHERE       numdep = ndepto
  GROUP BY   nomedep;
... //consultas sobre Info_Depto
DROP TABLE Info_Depto /* no final do serviço */
```

Comando DELETE

```
DELETE FROM Empregado  
WHERE      numat = '1234';
```

```
DELETE FROM Empregado  
WHERE      nomefim = 'Boulos';
```

```
DELETE FROM Empregado;
```

```
DELETE FROM Empregado  
WHERE      ndepto IN  
            (SELECT      numdepto  
              FROM        Departamento  
              WHERE        nomedep = 'Pesquisa');
```

Comando UPDATE

- Para modificar valores de atributos de uma ou mais tuplas selecionadas, cláusula SET adicional especifica atributos que serão modificados e seus valores.

```
UPDATE      Projeto
SET         locproj = 'Terrenos', ndep = 5
WHERE numproj = 10;
```

- Várias tuplas:

```
UPDATE Empregado
SET     salario = salario * 1.1
WHERE  ndepto IN (SELECT      numdep
                  FROM        Departamento
                  WHERE        nomedep = 'Pesquisa');
```

- Cada comando de atualização só se aplica a uma tabela

Comando SELECT

- Comando básico para recuperação de informação
- Comando SELECT de SQL não corresponde à operação de seleção da álgebra relacional
- Modelo Relacional X SQL:
 - No modelo relacional as entidades são tratadas como um conjunto, sem os elementos com os mesmos valores
 - SQL trata uma tabela não como um conjunto, logo registros duplicados podem aparecer no resultado da consulta
 - Para restringir tabelas a conjuntos: `DISTINCT`

Comando SELECT Básico

```
SELECT  <lista de atributos>  
FROM    <lista de tabelas>  
WHERE   <condição>
```

<lista de atributos> são os nomes dos atributos cujos valores devem ser recuperados pela consulta

<lista de tabelas> são os nomes das tabelas exigidos para o processamento da consulta

<condição> é uma expressão condicional que identifica que registros serão recuperados pela consulta

Empregados que trabalham no departamento 5

$\sigma_{\text{ndeppto} = 5}(\text{Empregado})$

```
SELECT *  
FROM   Empregado  
WHERE  ndeppto = 5;
```

Empregados do sexo feminino que trabalham no departamento 5

$\sigma_{\text{sexo} = 'F' \wedge \text{ndepto} = 5} (\text{Empregado})$

```
SELECT *  
FROM   Empregado  
WHERE  sexo = 'F' AND ndepto = 5;
```

Nome, número de matrícula e salário dos empregados

π nominic, nomefim, numat, salario (Empregado)

```
SELECT nominic, nomefim, numat, salario  
FROM    Empregado;
```

Lista de salário dos empregados

π_{salario} (Empregado)

```
SELECT salario  
FROM Empregado;
```

```
SELECT ALL salario  
FROM Empregado
```

Os salários repetidos aparecem, com ou sem a cláusula ALL

```
SELECT DISTINCT salario  
FROM Empregado;
```

Elimina as repetições

- Se a cláusula DISTINCT vier acompanhada de mais de um atributo DISTINCT aplica-se a cada registro como um todo, isto é, serão mantidos apenas os registros não repetidos para o grupo de atributos envolvidos.

Matrícula e salário dos empregados do departamento 5 que recebem até 300

$\pi_{\text{numat, salario}} (\sigma_{\text{ndepto} = 5 \wedge \text{salario} \leq 300} (\text{Empregado}))$

```
SELECT numat, salario  
FROM Empregado  
WHERE ndepto=5 AND salario<=300;
```

Matrícula e salário dos empregados do departamento 5 ou do sexo feminino

$\pi_{\text{numat, salario}} (\sigma_{\text{ndepto} = 5 \vee \text{sexo} = 'F'} (\text{Empregado}))$

```
SELECT numat, salario  
FROM    Empregado  
WHERE   ndepto=5 OR sexo='F' ;
```

Faça a junção (de igualdade) de projetos e departamentos que controlam estes projetos

Projeto ⋈_{ndep = numdep} Departamento

```
SELECT *  
FROM   Projeto, Departamento  
WHERE  ndep = numdep;
```

O que fazer para obter informações sobre empregados e sobre os projetos onde trabalham?

- Empregado(nominic, nomint, nomefim, numat, dtnasc, ender, sexo, salario, nsuper, ndepto)
- TrabalhaEm (numemp, nproj, horas)
- Projeto (nomeproj, numproj, locproj, ndep)

Empregado ⋈_{numat=numemp} TrabalhaEm ⋈_{nproj=numproj} Projeto

```
SELECT *  
FROM   Empregado, TrabalhaEm, Projeto  
WHERE  numat=numemp AND nproj = numproj;
```


Nome e salário dos empregados que estão lotados no Departamento de Pesquisa

$\pi_{\text{nominic, nomint, nomefim, salario}} (\sigma_{\text{nomedep} = \text{'Pesquisa'}} ($
 $\text{Empregado} \bowtie_{\text{ndepto} = \text{numdep}} \text{Departamento}))$

```
SELECT nominic, nomint, nomefim, salario
FROM    Empregado, Departamento
WHERE   ndepto=numdep AND nomedep='Pesquisa';
```

Para cada projeto locado em Terenos liste o número do projeto, número do departamento controlador e último nome do chefe

$\pi_{\text{numproj, ndep, nomefim}} (\sigma_{\text{locproj} = \text{'Terenos'}} (\text{Projeto} \bowtie_{\text{ndep} = \text{numdep}} \text{Departamento} \bowtie_{\text{nchefe} = \text{numat}} \text{Empregado}))$

```
SELECT numproj, ndep, nomefim
FROM    Projeto, Departamento, Empregado
WHERE   ndep=numdep AND nchefe=numat
        AND locproj='Terenos' ;
```

Nomes dos empregados que trabalham nos projetos controlados pelo departamento 5

$\pi_{\text{nominic, nomefim}}(\sigma_{\text{ndep}=5}(\text{Emp} \bowtie_{\text{numat=numemp}} \text{TrabalhaEm} \bowtie_{\text{nproj=numproj}} \text{Proj}))$

```
SELECT nominic, nomint, nomefim
FROM   Empregado, TrabalhaEm, Projeto,
WHERE  numat=numemp AND nproj=numproj
AND ndep=5;
```

Nomes de atributos ambíguos (Prefixação)

O nome do empregado, o nome do dependente e o sexo do dependente para empregados dos sexo masculino.

```
SELECT nominic, nomefim, depend,  
        Dependente.sexo  
FROM    Empregado, Dependente  
WHERE   numat = numemp AND  
        Empregado.sexo = 'M'
```

Nomes de atributos ambíguos (Aliases-apelidos)

O nome do empregado, o nome do dependente e o sexo do dependente para empregados dos sexo masculino.

```
SELECT E.nominic, E.nomefim, D.depend, D.sexo  
FROM   Empregado AS E, Dependente AS D  
WHERE  E.numat = D.numemp AND E.sexo = 'M'
```

Nomes dos empregados e nomes de seus supervisores diretos.

$E \leftarrow \pi_{\text{nominic, nomefim, numat, nsup}} (\text{Empreg})$

$S \leftarrow \pi_{\text{nominic, nomefim, numat, nsup}} (\text{Empreg})$

$\pi_{E.\text{nominic}, E.\text{nomefim}, S.\text{nominic}, S.\text{nomefim}} (E \bowtie_{E.\text{nsup}=S.\text{numat}} S)$

```
SELECT E.nominic, E.nomefim,  
       S.nominic, S.nomefim  
FROM   Empregado AS E, Empregado AS S  
WHERE  E.nsup = S.numat;
```

Conjuntos explícitos

```
SELECT DISTINCT numemp  
FROM   TrabalhaEm  
WHERE  nproj IN (1,2,3) ;
```

Consulta a valores NULL

```
SELECT nominic, nomefim  
FROM    Empregado  
WHERE   nsuper IS NULL;
```


Cláusula WHERE não especificada

```
SELECT nominic, nomefim, depend  
FROM    Empregado, Dependente
```

Combina cada nome do empregado com todos nomes de dependentes,
sem relação de qualificação, no resultado da consulta:
PRODUTO CARTESIANO

Sintaxe do comando SELECT

```
SELECT <lista de campos>  
FROM <lista de tabelas>  
[WHERE <condição>]  
[GROUP BY <campos que definem o grupo>  
[HAVING <condição sobre a função de grupo>] ]  
[ORDER BY <campos do resultado da consulta>]
```

Ordenando o resultado: ASC/DESC

```
SELECT nominic, salario  
FROM    Empregado  
WHERE   ndepto=5 OR sexo='F';  
ORDER BY salario, nominic
```

```
SELECT nominic  
FROM    Empregado  
ORDER BY nominic DESC
```

```
SELECT nominic  
FROM    Empregado  
ORDER BY nominic ASC
```

```
SELECT salario, nominic  
FROM    Empregado  
ORDER BY salario DESC, nominic ASC
```